

Guía de Instalación Local e Infraestructura de Supabase

Despliegue autogestionado (Self-Hosted) sin dependencias de suscripción

Este instructivo técnico detalla el procedimiento para descargar, configurar y ejecutar una instancia completa de **Supabase** en entornos locales o servidores propios de forma 100% gratuita, eliminando la dependencia de planes de pago comerciales y manteniendo el control absoluto sobre los datos.

1. Requisitos Previos del Sistema

Antes de iniciar, asegúrese de contar con las siguientes herramientas instaladas y activas en su entorno:

- **Docker Engine** (v20.10.0 o superior) y **Docker Compose** (v2.0.0 o superior) en ejecución.
- **Git** instalado para la clonación de repositorios de configuración.
- **Node.js & npm** (Recomendado para la gestión de migraciones y CLI).

2. Requisitos Mínimos y Recomendados de Hardware

Dado que Supabase levanta un ecosistema complejo de microservicios concurrentes (Postgres, GoTrue, Kong, Realtime, Studio, etc.), se deben garantizar los siguientes recursos de hardware en el servidor objetivo:

Componente	Mínimo Absoluto (Desarrollo / Bajo Tráfico)	Recomendado (Producción Inicial / PWA activa)
CPU	1 Núcleo (vCPU) x86_64 o ARM64 (ej. Apple Silicon / Raspberry Pi 4+)	2 a 4 Núcleos Dedicados
Memoria RAM	2 GB RAM (Con swap activo si es Linux)	4 GB a 8 GB RAM
Almacenamiento	10 GB libres de espacio en disco	20 GB+ en estado sólido (SSD / NVMe)
Sistema Operativo	Cualquier distribución Linux (Ubuntu 20.04+, Debian), macOS o Windows 10/11 con WSL2	Ubuntu Server 22.04 LTS o Debian Stable optimizado

Nota crítica sobre la Memoria RAM: Aunque Postgres solo puede correr con muy pocos recursos, levantar la suite completa (incluyendo Supabase Studio y Kong) en entornos con menos de 2 GB de RAM provocará el cierre inesperado de los contenedores por falta de memoria (Out-Of-Memory / OOM Killer).

3. Flujo de Trabajo Rápido (Entornos de Desarrollo)

Para levantar servicios de prueba, prototipado rápido o desarrollo de Aplicaciones Web Progresivas (PWAs) en su propia máquina, la interfaz de línea de comandos (CLI) de Supabase simplifica el proceso:

Paso 1: Instalación de la CLI de manera global

```
npm install -g supabase
```

Paso 2: Inicialización del proyecto

```
supabase init
```

Este comando generará una carpeta estructurada llamada `supabase/` que contiene las configuraciones locales de base de datos, políticas de seguridad y funciones del ecosistema.

Paso 3: Lanzamiento de los Contenedores

```
supabase start
```

La primera ejecución descargará las imágenes oficiales. Al finalizar, la consola desplegará las credenciales locales de desarrollo, los endpoints de las APIs y una URL de acceso para la administración visual.

Acceso Web Local (Supabase Studio): Por defecto, podrá administrar la base de datos relacional PostgreSQL, crear tablas y gestionar políticas ingresando desde su navegador a: <http://localhost:54323>

4. Despliegue en Servidores Propios (Producción)

Para entornos de producción permanente dentro de una red local o un servidor virtual privado (VPS), se utiliza el repositorio estructurado de auto-alojamiento.

Procedimiento de Despliegue:

1. Clone el repositorio de docker oficial:

```
git clone --depth 1 https://github.com/supabase/supabase.git
```

2. Acceda al directorio de configuración de Docker:

```
cd supabase/docker
```

3. Copie el archivo de variables de entorno de ejemplo:

```
cp .env.example .env
```

4. Edite el archivo `.env` (usando `nano .env`) para reemplazar obligatoriamente las claves por defecto (`JWT_SECRET` , passwords de Postgres, etc.) con valores fuertemente encriptados.
5. Inicie la infraestructura en modo persistente y desconectado:

```
docker compose up -d
```

5. Mapeo de Puertos y Servicios Principales

El ecosistema levantará contenedores independientes interconectados. Los accesos por defecto son:

Servicio Interno	Puerto Local	Descripción / Propósito
PostgreSQL	5432	Motor de base de datos relacional puro y soporte para extensiones (incluyendo AI/Vectors con <code>pgvector</code> , colas con <code>pgmq</code> y cron con <code>pg_cron</code>).
Kong API Gateway	8000	Punto de entrada único para APIs REST (vía PostgREST), GraphQL (vía <code>pg_graphql</code>), Auth y Realtime.
Supabase Studio	54323 / 3000	Panel de control web para la gestión e inspección de datos.
GoTrue (Auth)	9999	Gestor microservidor para autenticación, logins sociales y tokens JWT.
Storage & Edge Functions	Integrado	Almacenamiento local con políticas RLS y entorno de ejecución local para funciones con Deno.

6. Responsabilidades Operativas Críticas

- **Políticas de Backup:** Configure tareas programadas (ej. `cron` con `pg_dump`) para resguardar bases de datos fuera del servidor de manera automatizada.
- **Seguridad de Red (RLS):** Asegúrese de mantener activa la Seguridad por Filas (*Row Level Security*) en Postgres para proteger la API expuesta en el gateway `Kong` .
- **Actualizaciones de Software:** Monitoree de forma periódica los releases del repositorio oficial de Docker para actualizar las versiones de las imágenes ante vulnerabilidades detectadas.